

Sistema de Consulta Eficiente com Tabela Hash e Filtro de Bloom

Objetivo

Desenvolver um sistema capaz de armazenar e consultar grandes volumes de dados utilizando duas estruturas complementares:

- **Tabela Hash** → armazenamento e recuperação exata dos elementos;
- **Filtro de Bloom** → estrutura probabilística para acelerar consultas de existência.

O objetivo é analisar o impacto do uso combinado dessas estruturas em termos de:

- desempenho;
 - consumo de memória;
 - taxa de falsos positivos.
-

Tema do Projeto

Sistema de Verificação de Cadastro de Usuários

Imagine que uma empresa possui milhões de usuários cadastrados e deseja responder rapidamente perguntas como:

“Este usuário já está cadastrado?”

Para reduzir acessos desnecessários à estrutura principal, será utilizado um **Filtro de Bloom** antes da consulta na **Tabela Hash**.

Assim, na tarefa proposta, o sistema inicia carregando os usuários de um arquivo de texto para o filtro de Bloom e uma tabela HASH. Esse processo é realizado após execução do programa.

Uma vez em execução, toda vez que um novo elemento for inserido, deve ser inserido no filtro e na HASH.

Não será necessário implementar remoções.

Requisitos Funcionais

O sistema deverá possuir um *menu* que solicite as seguintes informações:

RF01 – Inserção

Cadastrar um novo usuário usando um identificador único.

Exemplo:

```
INSERIR joao123
INSERIR maria98
```

RF02 – Consulta

Consultar se um usuário existe.

Fluxo obrigatório:

1. Consultar o Filtro de Bloom;
2. Se o filtro indicar **“definitivamente não existe”**, retornar imediatamente;
3. Caso indique **“possivelmente existe”**, consultar a Tabela Hash;
4. Informar o resultado final.

Exemplo:

```
CONSULTAR joao123
→ Usuário encontrado

CONSULTAR ana777
→ Usuário inexistente
```

RF03 – Estatísticas

Exibir:

- quantidade de elementos armazenados;
 - quantidade de consultas realizadas;
 - consultas evitadas pela Bloom;
 - número de falsos positivos;
 - taxa percentual de falsos positivos;
 - tempo médio de consulta.
 - Outros
-

RF04 – Inserir em lote

Carregar registros a partir de arquivo texto.

Exemplo:

```
usuarios.txt
```

Conteúdo:

```
joao123  
maria98  
pedro45
```

Implementação Obrigatória

Parte 1 — Tabela Hash

Implementar manualmente:

- função hash;
- tratamento de colisão:
 - encadeamento externo **ou**
 - endereçamento aberto;
- operações:
 - inserção;
 - busca;

Parte 2 — Filtro de Bloom

Implementar manualmente:

- vetor de bits;
- Dimensionar adequadamente:
 - O tamanho do vetor de bits;
 - O falso positivo;
 - A quantidade de funções hash;
- inserção;
- consulta.

Parte 3 - Experimentos

Executar testes com conjuntos de:

- 1.000 registros;
- 10.000 registros;
- 100.000 registros.

Gere nomes aleatoriamente para poder povoar os arquivos no seguinte formato:

`[8caractere][3numeros]`

Exemplo:

```
islaifda122  
djksalsa297  
fjkldsaf881  
...
```

Para cada cenário medir:

Quantidade	Tempo sem Bloom	Tempo com Bloom	Falsos Positivos
1.000			
10.000			
100.000			

- Tempo sem bloom: Analisar quanto tempo é necessário para buscar todos os registros do arquivo sem utilizar o filtro de Bloom;
- Tempo com bloom: Analisar quanto tempo é necessário para buscar todos os registros do arquivo sem utilizar o filtro de Bloom;
- Tempo sem bloom: Analisar quanto tempo é necessário para buscar todos os registros do arquivo sem utilizar o filtro de Bloom
- Falsos Positivos: Porcentagem das vezes que o filtro de bloom disse que o estava presente mas, quando foi investigar pela tabela hash, não estava.

Responder:

1. O Filtro de Bloom reduziu consultas na Tabela Hash?
2. Como o tamanho do vetor impactou os resultados?
3. O número de funções hash alterou a precisão?
4. Em qual cenário o Bloom foi mais vantajoso?

Divisão da Equipe (3 integrantes)

Integrante 1 — Estrutura Hash

Responsável por:

- implementação da Tabela Hash;
- testes unitários;
- análise de colisões.

Entrega:

```
hash.*
```

Integrante 2 — Filtro de Bloom

Responsável por:

- implementação do Bloom;
- geração de hashes;
- medição de falsos positivos.

Entrega:

```
bloom.*
```

Integrante 3 — Integração e Avaliação

Responsável por:

- integração do sistema;
- leitura de arquivos;
- coleta de métricas;
- relatório experimental.

Entrega:

```
main.* + relatório
```

Entregáveis

1. Código-fonte completo;
2. Relatório (3–5 páginas);
3. Apresentação do trabalho (remotamente)
4. Arquivo **README.md** contendo:
 - instruções de compilação;
 - formato de entrada;
 - exemplos de execução.

Critérios de Avaliação

Critério	Peso
Correção da implementação	35%
Uso adequado das estruturas	25%
Qualidade experimental	20%
Organização do código	10%
Documentação	10%

Bônus (+10%)

Implementar uma das opções:

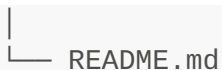
- comparação entre diferentes funções hash apresentados no relatório;
- Estudo do dimensionamento adequado do filtro de bloom apresentado no relatório.
- geração automática de gráficos de desempenho.

Restrições

- Não utilizar bibliotecas prontas de Tabela Hash ou Bloom Filter;
- Linguagem: **C**;
- Todo o código deve ser comentado;
- O relatório deve conter referências utilizadas.
- O relatório deverá conter as instruções de como obter, compilar, executar o programa.

Sugestão de Estrutura do Projeto

```
projeto/
├── src/
│   ├── hash.c
│   ├── hash.h
│   ├── bloom.c
│   ├── bloom.h
│   └── main.c
├── data/
│   └── usuarios.txt
├── testes/
└── relatorio.pdf
```



Observações finais:

A melhor implementação será aquela que apresentar as melhores justificativas técnicas para a escolha das dimensões da Tabela Hash e do Filtro de Bloom, e não necessariamente aquela que obtiver o menor tempo de execução.

Os grupos deverão demonstrar que as decisões de projeto foram tomadas com base em análise quantitativa e experimental.

Por exemplo:

- Escolher o tamanho da Tabela Hash considerando o fator de carga (load factor), explicando como ele afeta colisões e desempenho;
- Justificar o número de posições do vetor do Filtro de Bloom com base na quantidade esperada de elementos;
- Justificar a quantidade de funções hash utilizadas e seu impacto na taxa de falsos positivos;
- Comparar diferentes configurações antes de escolher a solução final.